

Prueba de Práctica 2: Conceptos Generales de Programación Avanzada

Programación Avanzada — Evaluación Autocontenida Integrada

Parte 1: Preguntas de Selección Múltiple

Pregunta 1. Inicialización de Generadores (Fase de Prime).

Dado un generador diseñado para actuar de forma interactiva recibiendo datos externos a través de su expresión `yield`, si ejecutamos de forma inmediata `generador.send(valor)` (donde `valor` no es `None`) justo después de instanciar el generador, ¿qué comportamiento se obtendrá?

- a) El valor inyectado se asignará a la primera variable y el generador continuará su ciclo.
- b) Se levantará una excepción del tipo `TypeError` indicando que no se puede enviar un valor a un generador recién nacido.
- c) El generador descartará el valor en silencio y retornará `None`.
- d) El programa se bloqueará de forma indefinida esperando que el hilo principal finalice.
- e) Se levantará la excepción `StopIteration` de inmediato.

Pregunta 2. Evaluación Perezosa y Expresiones Generadoras.

¿Cuál es la diferencia principal en el consumo de memoria entre una lista por comprensión y una expresión generadora al procesar un conjunto masivo de datos?

- a) La lista por comprensión es más eficiente en memoria porque almacena los elementos en un bloque contiguo optimizado.
- b) La expresión generadora genera y almacena la secuencia completa en la memoria caché del procesador antes de iniciar el recorrido.
- c) La lista por comprensión genera y almacena en memoria RAM todos los elementos de la secuencia al momento de su declaración, mientras que la expresión generadora evalúa y entrega cada elemento de manera perezosa (lazy) "uno a uno" a demanda, requiriendo un espacio de memoria mínimo y constante $\mathcal{O}(1)$.
- d) Las expresiones generadoras consumen más memoria RAM porque conservan todo el historial de llamadas en la pila de ejecución.
- e) No existe ninguna diferencia en el consumo de memoria; ambos son meros alias sintácticos.

Pregunta 3. Condición de Carrera y Locks.

Considere una variable numérica compartida accesible por varios hilos. Si realizamos incrementos simultáneos mediante la operación elemental $x += 1$ sin aplicar ningún mecanismo de sincronización, ¿qué anomalía puede ocurrir?

- a) Python lanzará de inmediato una excepción del tipo **ConcurrencyError**.
- b) La operación de incremento se ejecutará de forma atómica a nivel de hardware, por lo que el valor final siempre será exacto y correcto.
- c) Se producirá una condición de carrera (race condition) donde incrementos individuales se sobrescriben mutuamente, resultando en un valor final de x inferior al número total de incrementos teóricos.
- d) El intérprete de Python entrará en un ciclo infinito de espera (deadlock) intentando resolver el acceso.
- e) El valor final de x será superior al esperado debido a duplicación automática de subprocesos.

Pregunta 4. Hilos y Seguridad de Widgets en PyQt5.

Si estamos realizando una consulta pesada en una base de datos en segundo plano usando un hilo auxiliar de Python (**threading.Thread**), ¿por qué no es seguro actualizar directamente el texto de una etiqueta gráfica (**QLabel.setText()**) desde el código de ese hilo secundario?

- a) El compilador de PyQt5 detecta el acceso remoto y levanta un error del tipo **AccessViolation**.
- b) Los widgets de Qt no son seguros para hilos (not thread-safe); manipular componentes gráficos directamente desde hilos secundarios desestabiliza el bucle de eventos y suele resultar en cierres abruptos (segmentation faults) del programa.
- c) El hilo secundario entrará en espera (deadlock) de forma indefinida debido al Global Interpreter Lock de Python.
- d) La actualización se realiza correctamente pero los colores del widget se invierten de forma permanente.
- e) Un hilo secundario tiene prohibido invocar cualquier método de clases nativas de Python.

Pregunta 5. Layouts y Responsividad en Interfaces Gráficas.

¿Cuál es el beneficio de utilizar layouts de PyQt5 (**QVBoxLayout**, **QHBoxLayout**, **QGridLayout**) en lugar de posicionar los widgets mediante coordenadas fijas usando el método **setGeometry()**?

- a) Los layouts permiten usar estilos CSS personalizados, mientras que el posicionamiento por coordenadas no lo permite.
- b) El uso de layouts reduce el consumo de memoria de la tarjeta gráfica del computador.

- c) Los layouts automatizan el redimensionamiento, espaciado y alineación de los widgets de manera responsiva cuando la ventana se expande, se contrae o cambia de resolución.
- d) PyQt5 prohíbe el uso de coordenadas fijas y genera un error en tiempo de ejecución si se usa `setGeometry()`.
- e) Los layouts permiten saltarse el bucle de eventos para acelerar la inicialización de los componentes.

Parte 2: Preguntas de Desarrollo

Pregunta 6. Iteradores Personalizados Manuales (Materia: Excepciones e Iteradores).
Implemente un iterador personalizado llamado **FiltroPares** que actúe sobre un iterable de números enteros y entregue únicamente aquellos que sean pares.

- a) Defina la clase de modo que cumpla estrictamente el protocolo de iteración de Python, implementando los métodos especiales `__iter__` y `__next__` de forma manual.
- b) No debe duplicar los datos internamente en una lista intermedia para conservar eficiencia de memoria.
- c) Cuando no queden más elementos pares, el iterador debe levantar la excepción correspondiente para terminar la iteración de manera limpia.

Pregunta 7. Generación Cooperativa e Inyección de Datos (Materia: Programación Funcional).

Implemente un filtro de suavizado exponencial para una señal ruidosa de telemetría de sensores usando una corrutina basada en generadores.

- Defina una función generadora `suavizador(alpha: float)`.
- La función debe inicializar la variable de estado con el primer valor recibido.
- Posteriormente, debe entrar en un bucle infinito que entregue el valor suavizado actual mediante `yield` y reciba nuevos valores usando la estructura de asignación.
- La fórmula de suavizado es: $S_t = \alpha \cdot X_t + (1 - \alpha) \cdot S_{t-1}$, donde X_t es el dato nuevo y S_{t-1} el promedio suavizado anterior.
- Muestre un ejemplo de uso que incluya el paso obligatorio de activación ("prime").

Pregunta 8. Patrón Productor-Consumidor con Colas Limitadas (Materia: Threading y Concurrency).

Implemente un sistema básico de mensajería asíncrona entre hilos. Se debe contar con:

- Una cola de comunicación con una capacidad máxima de 3 elementos (`maxsize=3`).
- Un hilo Productor que envíe 5 mensajes de telemetría numéricos a la cola.
- Un hilo Consumidor que extraiga los mensajes y los procese (imprimir en consola).
- Mecanismo de parada limpia: El productor debe inyectar una señal centinela especial (`None`) al final del procesamiento para avisar al consumidor que debe salir de su bucle infinito y terminar.

Pregunta 9. Widgets Interactivos y Diseños de Caja (Materia: GUI PyQt5).
Diseñe una ventana básica que sirva como control de volumen numérico o contador interactivo. La aplicación debe:

- Heredar de `QWidget` y definir un layout de tipo caja vertical (`QVBoxLayout`) principal.
- Contener una etiqueta de texto (`QLabel`) que muestre el valor actual del contador (inicialmente 0) con alineación centrada.
- Contener una sub-caja horizontal (`QHBoxLayout`) que albergue dos botones de control: botón de incremento con el texto "+" y botón de decremento con el texto "-".
- Conectar los clics de los botones a slots que aumenten o disminuyan en 1 el valor del contador reflejado en la etiqueta de texto.

Solucionario

Respuestas de Selección Múltiple

Pregunta 1: b)

Explicación: Un generador no puede recibir un valor del tipo `send(valor)` (donde `valor` no sea `None`) en su estado inicial, dado que aún no ha alcanzado su primera expresión `yield` en el código. Para inicializarlo (hacer el "prime"), se debe avanzar primero hasta el primer `yield` llamando a `next(generator)` o a `generator.send(None)`.

Pregunta 2: c)

Explicación: Las listas por comprensión evalúan y crean la estructura completa en memoria inmediatamente al ser declaradas. Si el conjunto de datos es muy grande, esto puede agotar la memoria disponible. Por otro lado, las expresiones generadoras no almacenan los elementos en memoria, sino que actúan como generadores que producen cada elemento bajo demanda (evaluación perezosa o lazy), permitiendo procesar secuencias de tamaño infinito con una huella de memoria constante.

Pregunta 3: c)

Explicación: A pesar de escribirse como una sola línea en Python (`x += 1`), esta operación no es atómica a nivel de máquina ni de bytecode. Consta de tres fases diferenciadas: leer el valor actual de la memoria, sumarle una unidad en un registro de la CPU y escribir el nuevo valor de vuelta en memoria. Si un hilo es interrumpido por el planificador (scheduler) entre la lectura y la escritura, y otro hilo incrementa el valor en el intertanto, al reanudarse el primer hilo escribirá un valor desactualizado, sobrescribiendo y perdiendo el trabajo del segundo hilo. Este problema se soluciona protegiendo la zona crítica con un objeto de tipo `threading.Lock`.

Pregunta 4: b)

Explicación: Qt prohíbe de forma estricta interactuar con la interfaz gráfica o con sus widgets desde cualquier hilo que no sea el hilo principal de la GUI. Esto es debido a que las operaciones de dibujo en pantalla no están sincronizadas internamente a nivel de hilos para priorizar rendimiento. Si dos hilos manipulan el mismo widget simultáneamente, se producirá corrupción de memoria y crash de la aplicación. La comunicación desde un hilo de ejecución hacia la GUI debe realizarse de forma segura emitiendo una señal que sea capturada por el hilo principal.

Pregunta 5: c)

Explicación: El uso de coordenadas fijas (`setGeometry()`) hace que las interfaces sean rígidas. Si el usuario maximiza la ventana o cambia la escala del sistema operativo, los elementos se verán recortados o desplazados. Los administradores de diseño (layouts) de Qt calculan dinámicamente el tamaño mínimo, máximo e intermedio de cada widget secundario según las políticas de tamaño (`sizePolicy`) establecidas, adaptándose automáticamente a cualquier cambio en la geometría de la ventana padre.

Soluciones de Desarrollo

Pregunta 6 (Iteradores Personalizados Manuales):

```
class FiltroPares:
    def __init__(self, coleccion):
        # Guardamos el iterador interno de la colección original
        self.iterador_origen = iter(coleccion)

    def __iter__(self):
        # Un iterable debe retornar un objeto iterador.
        # En este caso, el objeto mismo es el iterador.
        return self

    def __next__(self):
        # Bucle que consume elementos del iterador origen hasta
        # encontrar uno par o agotar la secuencia
        while True:
            # next() sobre iterador_origen puede propagar StopIteration
            # lo que detendrá el bucle for de forma limpia
            elemento = next(self.iterador_origen)
            if elemento % 2 == 0:
                return elemento

# Demostración del uso
secuencia = [1, 3, 4, 7, 8, 10, 11]
iterador_pares = FiltroPares(secuencia)

print("Elementos pares encontrados:")
for num in iterador_pares:
    print(num) # Imprime: 4, 8, 10
```

Pregunta 7 (Generación Cooperativa e Inyección de Datos):

```
def suavizador(alpha: float):
    # a) Primer yield para recibir la lectura inicial
    valor_inicial = yield
    s_anterior = valor_inicial

    # b) Retornamos el valor inicial de forma directa
    yield s_anterior

    # c) Bucle infinito para recibir lecturas y actualizarlas
    while True:
        lectura = yield
        s_actual = alpha * lectura + (1.0 - alpha) * s_anterior
```

```

        s_anterior = s_actual
        # Entrega el resultado suavizado y suspende hasta el próx send()
        yield s_actual

# Demostración del uso
filtro = suavizador(alpha=0.3)

# Fase "Prime": Avanzar el generador al primer yield para que acepte datos
next(filtro)

# Enviar valor inicial
filtro.send(10.0)

# Simulación de telemetría con ruido
lecturas = [12.0, 15.0, 11.0, 18.0]
for valor in lecturas:
    promedio_filtrado = filtro.send(valor)
    print(f"Lectura: {valor:>4.1f} | Promedio Suavizado: {promedio_filtrado:.2f}")

```


Pregunta 8 (Patrón Productor-Consumidor con Colas Limitadas):

```
import threading
import queue
import time

def productor(cola_mensajes):
    for i in range(1, 6):
        time.sleep(0.05) # Simulación de recolección de sensor
        dato = f"Lectura-{i}"
        print(f"[Productor] Generado y encolado: {dato}")
        cola_mensajes.put(dato) # Bloqueante si la cola llega a maxsize

    # Envío del centinela de parada
    print("[Productor] Procesamiento completo. Enviando señal de parada.")
    cola_mensajes.put(None)

def consumidor(cola_mensajes):
    while True:
        # Espera bloqueante por un nuevo elemento
        mensaje = cola_mensajes.get()

        # Validación de señal centinela
        if mensaje is None:
            print("[Consumidor] Recibida señal de parada. Finalizando.")
            break

        print(f"[Consumidor] Procesando mensaje: {mensaje}")
        time.sleep(0.1) # Simulación de costo de procesamiento

# Instanciamos la cola con tamaño máximo limitado a 3 elementos
canal = queue.Queue(maxsize=3)

# Inicializamos los hilos correspondientes
hilo_prod = threading.Thread(target=productor, args=(canal,))
hilo_cons = threading.Thread(target=consumidor, args=(canal,))

hilo_prod.start()
hilo_cons.start()

hilo_prod.join()
hilo_cons.join()

print("[Sistema] Hilos apagados limpiamente de forma secuencial.")
```

Pregunta 9 (Widgets Interactivos y Diseños de Caja):

```
import sys
from PyQt5.QtWidgets import (QApplication, QWidget, QVBoxLayout,
                             QHBoxLayout, QLabel, QPushButton)
from PyQt5.QtCore import Qt

class ContadorInteractiva(QWidget):
    def __init__(self):
        super().__init__()
        self.valor = 0 # Estado de datos
        self.init_ui()

    def init_ui(self):
        self.setWindowTitle("Contador Interactiva")
        self.resize(250, 120)

        # Layout Principal Vertical
        layout_principal = QVBoxLayout()

        # Etiqueta indicadora de estado
        self.etiqueta_valor = QLabel(str(self.valor))
        # Centrar el texto en el widget
        self.etiqueta_valor.setAlignment(Qt.AlignCenter)
        layout_principal.addWidget(self.etiqueta_valor)

        # Layout Secundario Horizontal para botones
        layout_botones = QHBoxLayout()

        self.btn_menos = QPushButton("-")
        self.btn_menos.clicked.connect(self.decrementar)
        self.btn_mas = QPushButton("+")
        self.btn_mas.clicked.connect(self.incrementar)

        layout_botones.addWidget(self.btn_menos)
        layout_botones.addWidget(self.btn_mas)

        # Anidar el layout horizontal dentro del vertical principal
        layout_principal.addLayout(layout_botones)

        self.setLayout(layout_principal)

    def incrementar(self):
        self.valor += 1
        self.actualizar_etiqueta()
```

```
def decrementar(self):
    self.valor -= 1
    self.actualizar_etiqueta()

def actualizar_etiqueta(self):
    self.etiqueta_valor.setText(str(self.valor))

if __name__ == "__main__":
    app = QApplication(sys.argv)
    ventana = ContadorInteractiva()
    ventana.show()
    sys.exit(app.exec_())
```